

Vibe Coding Lab 4 — Chat History

Project: 33_missileCommand

Personal repository: Raghav-Hariharan/SE_VibeCoding_33_missileCommand

1. User

https://github.com/SETAPESU26/33_missileCommand

i have this project to do and this is my assigned github link tell me what i should do to complete it

2. Assistant

I checked the assigned repo and the lab handout. The assignment is a small Pygame project with one bug and three incomplete features. The required workflow is to clone/fork the assigned repo, run the Python game, record a 10-second before video, use prompts to fix the broken code and add the README features one by one, record another 10-second video, push to your personal repo, and keep separate commits for each task. Do not raise a PR to the main SETAPESU26 repo.

The four tasks were identified as:

1. Fix battery selection so only alive batteries with ammunition are considered.
2. Implement explosion_color(progress).
3. Implement on_city_destroyed(city).
4. Implement city_repair_threshold().

3. User

I ran python --version and Windows said Python was not found.

4. Assistant

We checked the Python launcher instead. The `py --version` command showed Python 3.13.15, so Python was already installed. We decided to use `py` commands for the project.

5. User

I installed pygame with `py -m pip install pygame`. `py -m pygame --version` printed pygame 2.6.1 and then a module execution message.

6. Assistant

Pygame installed successfully. The module execution message was not a problem. We then ran the actual game with `py game.py`.

7. User

The Missile Command game opened.

8. Assistant

We recorded the required before video before making any code changes.

9. User

done recording

10. Assistant

We started Task 1. The original function was:

```
def nearest_battery(self, target):  
    return min(self.batteries, key=lambda b: b.pos.distance_squared_to(target))
```

The problem was that it selected the geometrically closest battery without checking whether it was alive or had ammunition. We prompted the LLM to fix it while keeping the change limited to Task 1.

11. User

Task 1 prompt: Fix the battery selection bug so that nearest_battery only considers batteries that are alive and have ammunition, while still selecting the closest valid battery.

12. Assistant

We identified the edge case where all batteries are unavailable. The final Task 1 implementation was:

```
def nearest_battery(self, target):  
    available = [b for b in self.batteries if b.alive and b.ammo > 0]  
    if not available:  
        return None  
    return min(available, key=lambda b: b.pos.distance_squared_to(target))
```

launch() was also made to safely return when nearest_battery() returned None. The game was tested and ran normally.

13. User

Task 1 was committed as `Fix battery selection for unavailable batteries`.

14. Assistant

Task 1 was verified as a separate Git commit.

15. User

I couldn't initially find explosion_color(progress). Then I found:

```
def explosion_color(progress):  
    """Return an (r, g, b) colour for an explosion (progress 0..1 of its life), or None for the default."""  
    pass
```

16. Assistant

For Task 2 we prompted the LLM to implement an RGB transition from bright white/yellow at the beginning, through orange, toward red as the explosion fades, while safely clamping progress to 0.0–1.0.

17. User

Task 2 prompt: Implement explosion_color(progress) with a bright white/yellow -> orange -> red transition, handle slightly out-of-range progress, and keep it self-contained.

18. Assistant

The implementation used clamping and two linear interpolation stages. The user tested it and confirmed the explosion color changed visibly.

19. User

Task 2 was committed as `Add dynamic explosion colors`.

20. Assistant

Task 2 was verified as a separate Git commit.

21. User

The current Task 3 function was:

```
def on_city_destroyed(city):  
    """Called when a city is hit; add screen shake, sounds, or a game-over warning here."""  
    pass
```

22. Assistant

For Task 3 we chose a simple self-contained feedback implementation that prints the destroyed city's position and a clear warning, without adding external assets or modifying the Game class.

23. User

Task 3 prompt: Implement `on_city_destroyed(city)` with terminal feedback identifying the destroyed city's position, keep it self-contained, and do not add external dependencies.

24. Assistant

The implementation was:

```
def on_city_destroyed(city):  
    """Called when a city is hit; add screen shake, sounds, or a game-over warning here."""  
    print(f"WARNING: City at position {city.pos} has been destroyed!")  
    print("CITY DESTROYED!")
```

The game was run through the city-destruction state successfully, and Task 3 was committed separately as `Add city destruction feedback`.

25. User

The final unfinished Task 4 function was:

```
def city_repair_threshold():  
    """Return a score value at which a destroyed city is rebuilt, or None to disable city repair."""  
    pass
```

26. Assistant

For Task 4 the fixed threshold implementation was:

```
def city_repair_threshold():  
    """Return a score value at which a destroyed city is rebuilt, or None to disable city repair."""  
    return 2000
```

This supplies the existing repair bookkeeping with a 2000-point threshold without changing the repair logic itself.

27. User

Task 4 was committed as `Add city repair threshold`.

28. Assistant

All four tasks were now committed separately. Git history showed:

25925b1 Add city repair threshold

3fd811f Add city destruction feedback

7bff7b8 Add dynamic explosion colors

90f344f Fix battery selection for unavailable batteries

`git status` showed the working tree was clean.

29. User

The after video was recorded.

30. Assistant

We then prepared to push the completed project to the user's personal GitHub repository, rather than the SETAPESU26 repository.

31. User

Personal repository: https://github.com/Raghav-Hariharan/SE_VibeCoding_33_missileCommand.git

32. Assistant

We changed the `origin` remote to the personal repository and ran `git push -u origin main`. The push succeeded and the main branch was set to track the personal repository's origin/main.

33. User

The push succeeded.

34. Assistant

The completed project is now in the personal GitHub repository. The remaining lab submission items are the before video, after video, updated code/repo, and exported chat history. The lab handout explicitly lists videos, updated code, and chat history as deliverables.